

Microservices Architecture

Java Backend Interview — Short Notes

1. Monolith vs Microservices

	Monolith	Microservices
Structure	Single deployable unit — all modules in one codebase and process	Collection of small, independently deployable services
Deployment	Deploy entire app for any change	Deploy only the changed service
Scaling	Scale entire app — wasteful if only one module is bottleneck	Scale individual services independently
Tech stack	Single language/framework for everything	Polyglot — each service can use best-fit technology
Team org	All teams work on same codebase — merge conflicts, coordination overhead	One team owns one service — Conway's Law alignment
Failure isolation	Bug in one module can crash entire app	Service failure isolated — other services continue
Development speed	Fast initially, slows as codebase grows	Slower to set up, faster to iterate per team
Testing	Simpler — single process, no network calls	Complex — integration tests across services, contract testing
Latency	In-process calls — nanoseconds	Network calls — milliseconds. Distributed call overhead
Data management	Single shared database — easy transactions	Database per service — distributed transactions complex
Operational complexity	Low — one artifact, one process	High — service discovery, monitoring, tracing, orchestration
Good for	Small teams, early-stage products, simple domains	Large teams, complex domains, high scale requirements

✓ Don't start with microservices. Start monolith, identify bounded contexts under load, then extract. Premature decomposition is the #1 microservices mistake

■ Microservices trade code complexity for operational complexity. Only worth it at scale

2. Core Principles & Characteristics

- **Single Responsibility** — each service does one thing well. Aligned to a bounded context (DDD)
- **Independently deployable** — deploy, scale, and update each service without touching others
- **Loose coupling** — services communicate via well-defined APIs. No shared code/DB between services
- **High cohesion** — related functionality grouped in same service; unrelated functionality separated
- **Decentralised governance** — teams choose their own tech stack, DB, deployment per service
- **Designed for failure** — assume network calls fail. Implement timeouts, retries, circuit breakers
- **Database per service** — each service owns its data store. No shared DB between services
- **Smart endpoints, dumb pipes** — business logic in services; communication layer is just transport
- **Bounded Context (DDD)** — natural boundary for a microservice. Each context has its own domain model, language, and data

Bounded Context Example

```
Order Service → orders DB (order, orderItem, orderStatus)
User Service → users DB (user, address, preferences)
Payment Service → payments DB (payment, transaction, refund)
// 'Customer' means different things in each context – that's fine
```

3. Service Communication

3.1 Synchronous Communication

- **REST (HTTP/JSON)** — most common. Simple, human-readable, language-agnostic. Stateless
- **gRPC** — Google's RPC framework. Uses Protocol Buffers (binary), HTTP/2. 5-10x faster than REST. Strongly typed contracts via .proto files. Supports streaming (server/client/bidirectional). Best for internal service-to-service calls
- **GraphQL** — client specifies exact data shape. Reduces over-fetching/under-fetching. Good for API gateways aggregating multiple services

	REST	gRPC	GraphQL
Protocol	HTTP/1.1	HTTP/2	HTTP/1.1 or 2
Format	JSON/XML (text)	Protocol Buffers (binary)	JSON
Performance	Moderate	Fastest — binary, multiplexed	Moderate
Schema	OpenAPI/Swagger (optional)	Proto files (required)	GraphQL Schema (required)
Streaming	Limited (SSE/WebSocket)	Native 4 modes	Subscriptions
Best for	Public APIs, web clients	Internal services, high throughput	Frontend BFF, flexible queries
Browser	Native	Needs grpc-web proxy	Native

3.2 Asynchronous Communication

- **Message Brokers** — services communicate via messages/events. Decoupled — sender doesn't wait for receiver
- **Kafka** — distributed, high-throughput event streaming. Durable (messages stored on disk). Replay supported. Topics/partitions. Use for event sourcing, audit log, high-volume events
- **RabbitMQ** — traditional message broker. AMQP protocol. Exchanges + queues + bindings. Rich routing. Use for task queues, work distribution, RPC-style async
- **AWS SQS/SNS** — managed messaging. SQS = queue (pull). SNS = pub/sub (push). Fan-out: SNS → multiple SQS queues

```
// Sync: Order Service → HTTP → Payment Service (waits for response)
// Async: Order Service → Kafka 'order.created' → Payment Service consumes
// Order Service continues immediately – doesn't wait
```

	Synchronous	Asynchronous
Coupling	Temporal coupling — both must be up	Decoupled — services can be offline
Response	Immediate — caller waits	No immediate response — eventual
Error handling	Propagates to caller immediately	Dead letter queues, retry policies
Complexity	Simpler — request/response model	More complex — eventual consistency
Throughput	Limited by slowest service	High — decouple producer from consumer speed
Use when	Need immediate result (auth, payment confirm)	Events, notifications, background work, high volume

4. API Gateway

- Single entry point for all client requests. Routes requests to appropriate services
- **Responsibilities:** routing, authentication/authorisation, rate limiting, SSL termination, request/response transformation, load balancing, caching, logging
- **Backends for Frontends (BFF)** — separate gateway per client type (mobile, web, partner) — each optimises for its consumer

Feature	Description	Tool/Example
Routing	Route <code>/orders/*</code> → Order Service, <code>/users/*</code> → User Service	Spring Cloud Gateway, Kong, AWS API GW
Authentication	Verify JWT/OAuth2 token at gateway — services trust gateway	OAuth2 Resource Server, JWT validation
Rate Limiting	Throttle clients — 100 req/sec per API key	Redis-backed rate limiter, Kong rate-limit plugin
SSL Termination	Handle HTTPS at gateway — services use HTTP internally	Nginx, AWS ALB, Gateway
Request Transform	Add/remove headers, transform request/response body	Spring Cloud Gateway filters
Circuit Breaking	Stop routing to unhealthy downstream services	Resilience4j, Hystrix
Aggregation	Combine responses from multiple services into one	GraphQL BFF, custom aggregation
Logging/Tracing	Inject correlation/trace IDs into all requests	Zipkin, Jaeger, OpenTelemetry

```
// Spring Cloud Gateway route example
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE # lb = load balanced via Eureka
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
            - name: CircuitBreaker
          args: { name: orderCB, fallbackUri: forward:/fallback }
```

5. Service Discovery & Load Balancing

5.1 Service Discovery

- Services register themselves and discover others dynamically — no hard-coded IPs/ports
- **Client-side discovery** — client queries registry, picks instance, calls directly. e.g., Eureka + Ribbon/Spring LoadBalancer
- **Server-side discovery** — client calls load balancer; LB queries registry and routes. e.g., AWS ALB + ECS service discovery, Kubernetes

Tool	Type	Description	Used With
Eureka	Client-side	Netflix OSS. Services register; clients cache registry. Self-preservation mode on network partition	Spring Cloud Netflix

Consul	Both	HashiCorp. Health checks, KV store, multi-datacenter, DNS-based discovery. More features than Eureka	Spring Cloud Consul
Kubernetes DNS	Server-side	K8s built-in. Services resolved by DNS name (servicename.namespace.svc.cluster.local). kube-proxy routes	Any app on K8s
AWS Cloud Map	Server-side	Managed service registry for AWS. Integrates with ECS, EKS, Route 53	AWS microservices
Zookeeper	Client-side	Coordination service — service registry, distributed config, leader election	Kafka, older Spring Cloud

```
// Eureka service registration (Spring Boot)
@SpringBootApplication @EnableEurekaClient
public class OrderServiceApp { ... }
// application.yml
eureka.client.serviceUrl.defaultZone: http://eureka-server:8761/eureka/
```

5.2 Load Balancing

- **Client-side LB** — Spring Cloud LoadBalancer (replaces Ribbon). Algorithms: Round Robin (default), Random, WeightedResponse
- **Server-side LB** — AWS ALB/NLB, Nginx, HAProxy, Kubernetes Service
- **Sticky sessions** — route same client to same instance (cookie-based). Avoid in stateless services

```
// RestTemplate with load balancing
@Bean @LoadBalanced
public RestTemplate restTemplate() { return new RestTemplate(); }
// Use service name – Spring LB resolves to actual instance
restTemplate.getForObject("http://ORDER-SERVICE/orders/1", Order.class);
// WebClient with load balancing (reactive)
@Bean @LoadBalanced
public WebClient.Builder webClientBuilder() { return WebClient.builder(); }
```

6. Resilience Patterns

6.1 Circuit Breaker (Resilience4j)

- Prevents cascading failures — stops calling a failing service, returns fallback immediately
- **Three states:**
 - **CLOSED** — normal operation. Requests pass through. Counts failures
 - **OPEN** — failure threshold exceeded. All requests fail fast with fallback. No calls to downstream
 - **HALF_OPEN** — after wait duration, allow limited requests. If they succeed → CLOSED. If fail → OPEN

```
// Resilience4j Circuit Breaker
resilience4j.circuitbreaker:
instances:
orderService:
slidingWindowSize: 10 # last 10 calls
failureRateThreshold: 50 # 50% failures → OPEN
waitDurationInOpenState: 10s # wait 10s before HALF_OPEN
permittedNumberOfCallsInHalfOpenState: 3

@CircuitBreaker(name = "orderService", fallbackMethod = "fallbackOrder")
public Order getOrder(Long id) { return orderClient.getOrder(id); }
public Order fallbackOrder(Long id, Exception e) {
```

```
return Order.defaultOrder(id); // return cached/default response
}
```

6.2 Retry Pattern

- Automatically retry failed calls — useful for transient failures (network blip, brief service restart)
- **Exponential backoff with jitter** — avoid thundering herd. Wait 1s, 2s, 4s... + random jitter
- Retry on specific exceptions — not on business errors (400 Bad Request should not retry)

```
resilience4j.retry:
instances:
orderService:
maxAttempts: 3
waitDuration: 500ms
enableExponentialBackoff: true
exponentialBackoffMultiplier: 2
retryExceptions: [java.io.IOException, java.net.SocketTimeoutException]
ignoreExceptions: [com.example.BusinessException]
```

6.3 Bulkhead Pattern

- Isolate resources per service call — failures in one don't exhaust resources for others
- **Thread pool bulkhead** — dedicated thread pool per downstream service
- **Semaphore bulkhead** — limit concurrent calls per downstream service
- Example: Payment service calls Bank API (slow) and Fraud API (fast). Bulkhead gives each separate thread pool — slow Bank API can't starve Fraud API calls

```
resilience4j.bulkhead:
instances:
bankApi: { maxConcurrentCalls: 5, maxWaitDuration: 100ms }
fraudApi: { maxConcurrentCalls: 20, maxWaitDuration: 10ms }
```

6.4 Rate Limiter

- Limit number of calls to a service in a time window — protect downstream from overload
- Client-side: limit how many calls your service makes to another. Server-side: limit incoming requests at API gateway

```
resilience4j.ratelimiter:
instances:
externalApi:
limitForPeriod: 10 # max 10 calls
limitRefreshPeriod: 1s # per second
timeoutDuration: 500ms # wait up to 500ms for permit
```

6.5 Timeout Pattern

- Set maximum wait time for any service call. Prevents thread starvation waiting on slow services
- Set at multiple levels: connection timeout, read timeout, overall call timeout
- Rule of thumb: timeout should be < upstream service's own timeout (give time to handle error)

```
resilience4j.timelimiter:
instances:
orderService: { timeoutDuration: 3s, cancelRunningFuture: true }
```

Pattern	Problem it Solves	Key Config	Resilience4j Annotation
Circuit Breaker	Cascading failure — stop calling failed service	failureRateThreshold, waitDuration	@CircuitBreaker

Retry	Transient failures — network blip, brief outage	maxAttempts, waitDuration, backoff	@Retry
Bulkhead	Resource exhaustion — isolate service failures	maxConcurrentCalls	@Bulkhead
Rate Limiter	Overloading downstream services	limitForPeriod, refreshPeriod	@RateLimiter
Timeout	Slow service threads starvation	timeoutDuration	@TimeLimiter
Fallback	Graceful degradation — return default	fallbackMethod	fallbackMethod param

7. Observability: Tracing, Metrics & Logging

7.1 Distributed Tracing

- Track a request as it flows through multiple services — correlate logs and measure per-service latency
- **Trace** — end-to-end journey of one request. **Span** — one unit of work within a trace (one service call)
- **Trace ID** — same across all services for one request. **Span ID** — unique per service/operation
- **OpenTelemetry** — CNCF standard for tracing, metrics, logs. Vendor-neutral. Replaces Sleuth + Zipkin in Spring Boot 3+
- **Jaeger / Zipkin** — tracing backends. Collect spans, show trace waterfall, identify slow services

```
// Spring Boot 3 – Micrometer Tracing (OpenTelemetry)
// pom.xml: micrometer-tracing-bridge-otel + opentelemetry-exporter-zipkin
management.tracing.sampling.probability: 1.0 # 100% in dev, 0.1 in prod
// Trace ID automatically propagated in HTTP headers: traceparent / X-B3-TraceId
```

7.2 Centralized Logging

- Aggregate logs from all services into one searchable system
- **ELK Stack** — Elasticsearch (store/search) + Logstash (ingest/transform) + Kibana (visualise)
- **EFK Stack** — Elasticsearch + Fluentd + Kibana. Fluentd is lighter than Logstash
- **Loki + Grafana** — label-based log aggregation. Cheaper than ELK. Good with Kubernetes
- Always log with **correlation ID / trace ID** — filter all logs for one request across services

```
// Structured JSON logging – machine readable
{"timestamp":"2026-01-01T10:00:00Z", "level":"ERROR",
"service":"order-service", "traceId":"abc123",
"message":"Order not found", "orderId":42}
```

7.3 Metrics

- **Micrometer** — Spring Boot's metrics facade. Supports Prometheus, Datadog, CloudWatch, InfluxDB
- **Prometheus + Grafana** — most common combo. Prometheus scrapes /actuator/prometheus endpoint. Grafana visualises
- **Key metrics:** request rate, error rate, latency (p50/p95/p99), JVM heap, thread pool size, circuit breaker state
- **RED method** — Rate, Errors, Duration — the three metrics every service should expose
- **USE method** — Utilisation, Saturation, Errors — for infrastructure metrics

7.4 Health Checks

- **Spring Actuator /actuator/health** — exposes health status. Integrates with K8s liveness/readiness probes
- **Liveness probe** — is the process alive? Restart if fails. /actuator/health/liveness

- **Readiness probe** — is service ready to accept traffic? Remove from LB if fails. /actuator/health/readiness
- **Startup probe** — is service done starting? Delays liveness checks during slow startup

```
management.endpoint.health.probes.enabled: true
```

```
management.health.livenessState.enabled: true
```

```
management.health.readinessState.enabled: true
```

8. Event-Driven Architecture

8.1 Events vs Commands vs Queries

Type	Description	Example	Direction
Command	Intent to perform action — may fail. Has one handler	PlaceOrder, ProcessPayment	One-to-one
Event	Fact that happened — cannot be undone. Multiple consumers OK	OrderPlaced, PaymentProcessed	One-to-many
Query	Request for data — no side effects	GetOrderById, FindUserByEmail	One-to-one

8.2 Kafka Core Concepts

- **Topic** — named stream of records. Partitioned for parallelism
- **Partition** — ordered, immutable sequence of records. Parallelism unit. More partitions = more consumers
- **Offset** — position of record in partition. Consumer tracks its offset
- **Producer** — writes records to topics. Key-based partitioning for ordering per key
- **Consumer Group** — group of consumers sharing work. Each partition consumed by one consumer in group
- **Broker** — Kafka server. Cluster = multiple brokers. Leader per partition
- **Retention** — messages stored for configurable time (7 days default). Consumers can replay
- **Exactly-once semantics** — idempotent producer + transactional API ensures no duplicates

```
// Spring Kafka producer
@Autowired KafkaTemplate kafkaTemplate;
kafkaTemplate.send("order.created", order.getId().toString(), event);

// Spring Kafka consumer
@KafkaListener(topics = "order.created", groupId = "payment-service")
public void handleOrderCreated(OrderEvent event) {
    paymentService.processPayment(event.getOrderId());
}
```

8.3 Outbox Pattern

- Guarantees atomicity between DB write and event publish — prevents lost events or dual writes
- **Problem:** save order to DB AND publish event to Kafka — what if Kafka fails after DB save? Order saved but event lost
- **Solution:** save order + event to OUTBOX table in same DB transaction. Separate relay process polls OUTBOX and publishes to Kafka

```
// 1. In same transaction:
orderRepo.save(order);
outboxRepo.save(new OutboxEvent("order.created", toJson(order)));

// 2. Separate Outbox relay (Debezium CDC or polling):
// Reads outbox table → publishes to Kafka → marks as published
```

✓ Debezium — CDC (Change Data Capture) tool that reads DB transaction log (WAL) and publishes changes to Kafka.
Zero-latency outbox pattern

9. Saga Pattern — Distributed Transactions

- Manages distributed transactions across multiple services without 2PC (Two-Phase Commit)
- Sequence of local transactions — each publishes event/message triggering next step
- **Compensating transactions** — if step N fails, run compensating actions for steps 1..N-1

9.1 Choreography Saga

- No central coordinator — services react to events and emit new events
- **Pros:** simple, no single point of failure, services fully decoupled
- **Cons:** hard to understand overall flow, difficult to debug, risk of cyclic dependencies

Order Service: OrderCreated →

Payment Service: ← OrderCreated → PaymentProcessed →

Inventory Service: ← PaymentProcessed → InventoryReserved →

Order Service: ← InventoryReserved → OrderConfirmed

// If payment fails: PaymentFailed → Order Service cancels order

9.2 Orchestration Saga

- Central Saga Orchestrator coordinates steps — tells each service what to do
- **Pros:** clear flow in one place, easy to monitor, handles complex logic
- **Cons:** orchestrator becomes coupled to all services, single point of failure risk

OrderSaga (Orchestrator):

1. → PaymentService.charge() → success → step 2

2. → InventoryService.reserve() → success → step 3

3. → ShippingService.schedule() → success → OrderConfirmed

// If step 2 fails:

→ PaymentService.refund() // compensating transaction

→ OrderService.cancelOrder()

	Choreography	Orchestration
Control	Distributed — each service decides	Central orchestrator controls flow
Coupling	Event-based — loosely coupled	Orchestrator knows all participants
Visibility	Hard — flow spread across services	Easy — all steps in orchestrator
Debugging	Difficult — trace events across logs	Easier — single place to check state
Tooling	Any message broker	Temporal, Axon, AWS Step Functions
Use for	Simple flows, few services	Complex flows, many steps, long-running

10. CQRS & Event Sourcing

10.1 CQRS — Command Query Responsibility Segregation

- Separate read model and write model — different data models optimised for each operation
- **Command side (write)** — handles state changes. Validates, applies business rules, persists. Normalised DB
- **Query side (read)** — handles data retrieval. Denormalised, optimised for queries. Separate read DB (Elasticsearch, Redis, read replica)
- **Sync mechanism** — events/CDC propagate write changes to read model. Eventual consistency
- **Benefits:** scale reads/writes independently, optimise each model, clear separation of concerns
- **Cost:** eventual consistency between read/write, more complex, two models to maintain


```
// Command: POST /orders → Order Service → validates → saves to PostgreSQL
// → publishes OrderCreated event → updates Elasticsearch read model
// Query: GET /orders?status=PENDING → hits Elasticsearch directly (fast!)
```

10.2 Event Sourcing

- Store state as sequence of events — never update/delete, only append
- Current state = replay all events from beginning. Snapshot for performance
- **Benefits:** full audit trail, time-travel (state at any point), easy event replay, natural CQRS fit
- **Cost:** eventual consistency, complex queries, snapshot management, schema evolution hard

```
// Traditional: UPDATE orders SET status='SHIPPED' WHERE id=1
// Event Sourced: INSERT events(orderId, type, data, timestamp) VALUES
// (1, 'OrderPlaced', {...}, t1)
// (1, 'PaymentMade', {...}, t2)
// (1, 'OrderShipped', {...}, t3)
// Current state: replay events → OrderStatus = SHIPPED
```

✓ CQRS and Event Sourcing are independent — you can use one without the other. But they complement each other well

11. Security in Microservices

11.1 Authentication & Authorisation Patterns

- **JWT at API Gateway** — validate token at gateway; pass claims as headers to services. Services trust gateway (mTLS or internal network)
- **OAuth2 / OIDC** — industry standard. Auth Server issues tokens (Keycloak, Auth0, Cognito). Services validate tokens
- **mTLS (mutual TLS)** — service-to-service auth using certificates. Each service presents client cert. Prevents rogue services
- **Service mesh** — Istio/Linkerd handle mTLS automatically between all services

```
// Gateway validates JWT → extracts user-id, roles
// Passes as headers to downstream:
// X-User-Id: 42, X-User-Roles: ADMIN,USER
// Services trust these headers (only gateway can set them)
```

11.2 Service-to-Service Auth

- **Client Credentials flow (OAuth2)** — service authenticates as itself to get access token, then calls another service
- **API Keys** — simple but less secure. Suitable for internal networks
- **mTLS** — strongest. Certificate-based. Managed by service mesh (Istio)

```
// Spring Security OAuth2 client credentials
spring.security.oauth2.client.registration.order-service:
client-id: order-service
client-secret: secret
authorization-grant-type: client_credentials
```

11.3 Security Best Practices

- Never trust service-to-service calls without authentication (zero-trust network model)
- Rotate secrets regularly — use Vault or AWS Secrets Manager, not hardcoded env vars
- Least privilege — each service has minimum permissions needed
- Encrypt data in transit (TLS) and at rest
- Sanitize inputs at each service boundary — don't assume upstream validated

12. Data Management Patterns

12.1 Database per Service

- Each service owns its data — no shared DB. Choose DB per service based on needs
- **Polyglot persistence** — User Service: PostgreSQL; Product Service: MongoDB; Session: Redis; Search: Elasticsearch
- **Data duplication** — acceptable. Keep only needed data in each service. Sync via events
- **No cross-service DB joins** — join in application layer or API Composition pattern

12.2 Handling Distributed Data

Pattern	Description	Use Case
API Composition	Call multiple services, join results in application/gateway	Simple read queries spanning services
CQRS Read Model	Denormalised read DB updated by events — pre-joined data	Complex reporting across services
Saga	Coordinate writes across services via events/orchestrator	Multi-service transactions
Outbox Pattern	Atomic DB + event publish using outbox table + CDC	Reliable event publishing
Shared Data Service	Reference data service (currencies, categories) consulted by others	Static/slow-changing shared data
Data Replication	Sync copies of needed data via events (e.g., user name in Order Service)	Avoid cross-service calls for read-only data

■ Eventual consistency is the norm in microservices. Design UX to handle it — optimistic updates, loading states, eventual refresh

13. Deployment — Docker & Kubernetes

13.1 Docker Essentials

- Package service + runtime + dependencies into portable container image

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/order-service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- **Multi-stage build** — build in one stage, copy artifact to slim runtime image

```
FROM maven:3.9-eclipse-temurin-21 AS build
COPY . . && RUN mvn package -DskipTests
FROM eclipse-temurin:21-jre-alpine
COPY --from=build /target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

13.2 Kubernetes Key Concepts

K8s Object	Description	Microservices Use
Pod	Smallest deployable unit — one or more containers sharing network/storage	Runs one service instance
Deployment	Manages Pod replicas, rolling updates, rollbacks	Deploy service with N replicas

Service	Stable network endpoint for pods. Types: ClusterIP, NodePort, LoadBalancer	Internal/external service discovery
ConfigMap	Store non-sensitive config as key-value pairs	application.properties per environment
Secret	Store sensitive data (base64 encoded, encrypted at rest)	DB passwords, API keys, TLS certs
Ingress	HTTP routing to Services — path-based, host-based. Nginx/Traefik	API gateway / reverse proxy
HPA	Horizontal Pod Autoscaler — auto-scale pods on CPU/memory/custom metrics	Auto-scale under load
Namespace	Logical isolation within cluster	Separate dev/staging/prod, per team
StatefulSet	For stateful apps — stable pod identity, persistent storage	Kafka, Zookeeper, databases on K8s
PersistentVolume	External storage mounted into pods — survives pod restarts	DB data persistence

13.3 Service Mesh (Istio/Linkerd)

- Infrastructure layer handling service-to-service communication — sidecar proxy (Envoy) alongside each pod
- **Features:** mTLS automatically, traffic management (canary, A/B), circuit breaking, retries, timeouts, distributed tracing, metrics — all without code changes
- **Istio:** feature-rich but complex. **Linkerd:** lightweight, simpler. **AWS App Mesh:** managed on AWS
- **Trade-off:** powerful but adds latency (~1ms per hop) and operational complexity

14. Spring Cloud — Microservices Toolkit

Module	Purpose	Key Features
Spring Cloud Gateway	API Gateway — routing, filters, security	Route predicates, GatewayFilter, circuit breaker integration, rate limiting with Redis
Spring Cloud Netflix Eureka	Service discovery — registry	Self-registration, client-side LB, self-preservation mode, dashboard
Spring Cloud LoadBalancer	Client-side load balancing (replaces Ribbon)	Round-robin, random, weighted strategies. @LoadBalanced RestTemplate/WebClient
Spring Cloud OpenFeign	Declarative HTTP client — annotated interfaces	@FeignClient, auto LB, Hystrix/Resilience4j integration, Feign interceptors
Spring Cloud Config	Centralised config server — Git, Vault backend	Environment-specific config, refresh without restart, encryption
Spring Cloud Sleuth/Micrometer	Distributed tracing — trace/span IDs	Auto propagation in HTTP headers, Kafka, gRPC. Exports to Zipkin/Jaeger/OTLP
Spring Cloud Stream	Event-driven messaging — Kafka/RabbitMQ abstraction	@EnableBinding, binders, functional model (Consumer/Supplier/Function beans)
Spring Cloud Kubernetes	Native K8s integration — ConfigMap, Service discovery	ConfigMap reload, pod metadata, K8s native service discovery

```
// Feign Client – declarative REST calls
@FeignClient(name = "payment-service", fallback = PaymentFallback.class)
public interface PaymentClient {
    @PostMapping("/payments")
```

```
PaymentResponse charge(@RequestBody PaymentRequest req);  
}
```

15. Frequently Asked & Tricky Interview Questions

Architecture Fundamentals

Q1. What are microservices? How do they differ from monolith?

→ Microservices: suite of small, independently deployable services each owning its data and functionality. Monolith: single deployable unit with all modules. Microservices give independent scaling, deployment, and tech choice at the cost of distributed systems complexity

Q2. When should you NOT use microservices?

→ Small teams (< 5-8 devs), early product with unknown boundaries, simple domain, limited operational maturity. Start monolith — identify high-traffic/change areas — then extract services. Premature decomposition creates distributed monolith with all downsides of both

Q3. What is a bounded context?

→ DDD concept — logical boundary where a domain model applies consistently. Natural unit for a microservice. 'Order' means different things in Order context vs Shipping context — they have separate models and data

Q4. What is Conway's Law? How does it apply?

→ Organizations design systems mirroring their communication structures. Microservices work best when team structure mirrors service boundaries — one team per service with full ownership (you build it, you run it)

Q5. What is a distributed monolith?

→ Multiple services that are tightly coupled — shared DB, synchronous calls everywhere, must deploy together. Worst of both worlds: distributed systems complexity without microservices benefits. Caused by improper decomposition or shared DB

Communication

Q6. When to use synchronous vs asynchronous communication?

→ Sync (REST/gRPC): need immediate response, user-facing operations (login, order placement confirmation). Async (Kafka/RabbitMQ): background processing, events, notifications, high-throughput, when caller doesn't need to wait

Q7. What is gRPC? When is it better than REST?

→ RPC framework using Protocol Buffers (binary) over HTTP/2. Better for: internal service calls needing high throughput, bidirectional streaming, strong typing with generated clients. Not ideal for public APIs or browser clients (needs grpc-web proxy)

Q8. What is the role of an API Gateway?

→ Single entry point for clients. Handles: routing, authentication, rate limiting, SSL termination, request transformation, load balancing. Decouples client from service topology changes — clients don't need to know service addresses

Q9. What is BFF (Backend for Frontend)?

→ Separate API gateway per client type (mobile, web, partner). Mobile BFF returns lightweight data; web BFF returns richer data. Avoids over/under-fetching for different clients and avoids one gateway trying to satisfy all client needs

Q10. How do you handle service-to-service communication failures?

→ Circuit breaker (stop calling failed service), retry with exponential backoff (transient failures), timeout (don't wait forever), fallback (return cached/default), bulkhead (isolate resource pools). Combine these with Resilience4j

Resilience

Q11. Explain the Circuit Breaker pattern. What are its states?

→ Prevents cascading failure. CLOSED: normal — tracks failures. OPEN: threshold breached — fail fast, return fallback. HALF_OPEN: after wait — test limited requests. Success → CLOSED; failure → OPEN. Resilience4j is standard Java implementation

Q12. What is the difference between Circuit Breaker and Retry?

→ Retry: good for transient failures — brief network blip. Retry a few times with backoff. Circuit Breaker: good for sustained failures — service down. Stop all calls immediately, give service time to recover. Often combined: retry within circuit breaker

Q13. What is the Bulkhead pattern?

→ Isolate resources per downstream service so failure in one doesn't exhaust resources for others. Like ship bulkheads — one flooding compartment doesn't sink ship. Implemented as separate thread pools or semaphores per service dependency

Q14. How do you handle timeouts in microservices?

→ Set connection timeout (how long to wait for connection), read timeout (how long to wait for response). Use Resilience4j TimeLimiter. Rule: timeout < upstream service's own timeout. Always have timeouts — never wait indefinitely on external calls

Q15. What is a fallback? Examples?

→ Alternative response when primary call fails. Examples: return cached data (last known value), return default/empty response, call secondary service, return partial response. Goal: graceful degradation rather than error propagation

Data & Transactions

Q16. Why database per service? What are the challenges?

→ Loose coupling — services don't share data models. Independent scaling. Choose best DB per service. Challenges: no cross-service joins (use API composition/CQRS), distributed transactions (use Saga), data consistency (eventual)

Q17. How do you handle distributed transactions?

→ Two-Phase Commit (2PC) — avoid: blocking, single point of failure. Saga pattern: sequence of local transactions with compensating actions on failure. Choreography (event-driven) or Orchestration (central coordinator)

Q18. What is the Saga pattern? Choreography vs Orchestration?

→ Saga: manages distributed transactions via local transactions + compensating actions. Choreography: services react to events autonomously — decoupled but hard to trace. Orchestration: central saga orchestrator directs each step — clear flow but coupled

Q19. What is the Outbox pattern? Why use it?

→ Solve dual-write problem: save to DB AND publish event atomically. Solution: save entity + event to outbox table in same DB transaction. Separate relay publishes outbox events to Kafka. Ensures no lost events even if Kafka is temporarily down

Q20. What is CQRS? When to use it?

→ Command Query Responsibility Segregation — separate write model (commands) and read model (queries). Use when: read and write performance needs differ, complex reporting queries, different scaling for reads vs writes. Cost: eventual consistency, more complex

Q21. What is Event Sourcing? Benefits and challenges?

→ Store state as sequence of events — never update, only append. Benefits: full audit trail, time-travel, event replay, natural fit with CQRS. Challenges: eventual consistency, complex queries, schema evolution, snapshot management for performance

Observability

Q22. What is distributed tracing? What is a Trace vs Span?

→ Track request across multiple services. Trace: end-to-end journey — unique Trace ID. Span: work in one service — Span ID. All spans for same request share Trace ID. Tools: Jaeger, Zipkin. Spring Boot 3: Micrometer Tracing (OpenTelemetry)

Q23. What is the ELK stack?

→ Elasticsearch (store/search logs) + Logstash (ingest/parse logs) + Kibana (visualise/query). Centralised logging for all services. Always log with correlation/trace ID to filter all logs for one request

Q24. What are liveness and readiness probes in Kubernetes?

→ Liveness: is container alive? Fail → K8s restarts container. Readiness: is container ready for traffic? Fail → K8s removes from Service endpoints — no traffic routed. Spring Actuator exposes /health/liveness and /health/readiness

Q25. What is the RED method for metrics?

→ Rate (requests per second), Errors (failed requests per second), Duration (latency distribution). The three key metrics every service should expose. Complement with USE method for infrastructure: Utilisation, Saturation, Errors

Security

Q26. How do you handle authentication in microservices?

→ Validate JWT at API Gateway — extract user ID and roles, pass as trusted headers to services. Services don't need to validate tokens — they trust gateway (protected by network policy/mTLS). Auth Server (Keycloak/Cognito) issues tokens

Q27. What is service-to-service authentication?

→ mTLS: certificate-based — strongest, managed by Istio/service mesh. OAuth2 Client Credentials: service gets token from Auth Server and passes in Authorization header. API Keys: simple but less secure. Never use no auth for internal services (zero-trust)

Q28. What is zero-trust security?

→ Never trust any request based on network location alone — verify every request. Even internal service calls require authentication. Assume breach — authenticate all service-to-service calls, encrypt all traffic (mTLS), least privilege access

Advanced & Patterns

Q29. What is service mesh? Why use it?

→ Infrastructure layer (Istio/Linkerd) handling service-to-service communication via sidecar proxies (Envoy). Provides: mTLS, traffic management (canary), circuit breaking, retries, tracing, metrics — without code changes. Trade-off: operational complexity

Q30. What is a canary deployment?

→ Gradually roll out new version to subset of users/traffic — e.g., 5% traffic to v2. Monitor metrics. If healthy, increase to 100%. If issues, rollback instantly. Managed by Istio, K8s, or AWS CodeDeploy. Reduces risk vs big-bang deployment

Q31. What is blue-green deployment?

→ Two identical production environments (blue=current, green=new). Deploy to green, test, then switch all traffic from blue to green. Instant rollback by switching back. Requires double infrastructure. Good for zero-downtime deploys

Q32. What is eventual consistency? How do you design for it?

→ Changes propagated asynchronously — all nodes will converge to same state, but not immediately. Design: optimistic UI updates, eventual refresh, show loading states, idempotent operations, compensating transactions for failures

Q33. How do you handle idempotency in microservices?

→ Idempotent: same operation called multiple times has same effect as once. Critical for retries. Implement: unique idempotency key per request (client sends, server checks), store processed keys in DB, check before processing. PUT/DELETE are naturally idempotent; POST needs explicit handling

Q34. What is API versioning in microservices?

→ Version APIs to allow independent evolution. Strategies: URL path (/api/v1/orders), header (Accept: application/vnd.app.v1+json), query param (?version=1). Support old versions during transition. Service mesh can route by version

Q35. How do you do service discovery without Eureka in Kubernetes?

→ K8s built-in DNS — each Service gets a DNS name: servicename.namespace.svc.cluster.local. Pods resolve service names automatically. No Eureka needed — K8s Service object provides stable virtual IP and DNS, kube-proxy routes to healthy pods

Q36. What is a sidecar pattern?

→ Deploy auxiliary container (sidecar) alongside main container in same pod. Handles cross-cutting concerns: logging agent, service mesh proxy (Envoy), config reload, TLS termination. Main container focuses on business logic only

Q37. How do you handle schema changes in event-driven systems?

→ Schema Registry (Confluent) enforces compatibility. Strategies: backward compatible (add optional fields, never remove), forward compatible, full compatible. Use Avro/Protobuf with versioned schemas. Never rename required fields

Q38. What is the Strangler Fig pattern?

→ Gradually migrate monolith to microservices — incrementally extract services behind a facade. Traffic routed to new service or monolith based on feature. Eventually monolith is fully 'strangled'. Low risk incremental migration

Q39. How do you test microservices?

→ Unit tests (each service in isolation), Integration tests (service + DB), Contract tests (Pact — verify API contracts between producer and consumer), Component tests (service with mocked dependencies), E2E tests (full flow, expensive, few). Shift-left — catch issues early

Q40. What is Pact/Consumer-Driven Contract Testing?

→ Consumer defines expected API contract (pact file). Provider verifies it passes against actual API. Catches breaking changes before deployment. Avoids expensive E2E tests for every change. Run in CI pipeline

Quick Fire

Q41. What is the difference between horizontal and vertical scaling?

→ Vertical (scale up): bigger machine — more CPU/RAM. Limited ceiling, single point of failure. Horizontal (scale out): more instances — add pods/servers. Microservices designed for horizontal scaling per service

Q42. What is a dead letter queue (DLQ)?

→ Queue where failed messages land after max retries. Prevents message loss. Alerts when messages in DLQ. Process manually or requeue after fix. Both Kafka (separate topic) and RabbitMQ/SQS have DLQ support

Q43. What is consumer group in Kafka?

→ Group of consumers sharing partition consumption. Each partition assigned to one consumer in group. More consumers than partitions → some idle. Use multiple groups to have multiple independent consumers of same topic

Q44. What is the difference between topic and queue?

→ Queue (SQS, RabbitMQ queue): message consumed by one consumer — work distribution. Topic (Kafka, SNS): message consumed by all subscribers independently — broadcast/pub-sub. Kafka actually allows both via consumer groups

Q45. How do you handle secret management in microservices?

→ Never hardcode secrets. Use: HashiCorp Vault (dynamic secrets, auto-rotation), AWS Secrets Manager, K8s Secrets (base64, encrypted at rest). Inject as env vars or files. Rotate regularly. Spring Cloud Vault integrates with Spring Boot

Q46. What is Kubernetes HPA?

→ Horizontal Pod Autoscaler — automatically scales pod count based on CPU/memory utilisation or custom metrics (Kafka lag, request queue depth). Scales out under load, scales in when quiet. Requires metrics-server

Q47. What is the difference between Dockerfile CMD and ENTRYPOINT?

→ ENTRYPOINT: always runs — the executable. CMD: default args to ENTRYPOINT, easily overridden. Best practice: ENTRYPOINT ["java"] CMD ["-jar","app.jar"]. Or ENTRYPOINT ["java","-jar","app.jar"] with no CMD

Q48. What is a ConfigMap in Kubernetes?

→ K8s object storing non-sensitive config as key-value pairs — mounted as env vars or files. Decouples config from image. Spring Cloud Kubernetes can watch ConfigMap and auto-refresh application properties without restart

Q49. What is idempotency key?

→ Client-generated unique ID for a request (UUID). Server stores processed keys. On retry with same key, server returns same response without reprocessing. Standard header: Idempotency-Key. Prevents duplicate payments, double-processing

Q50. What is the difference between orchestration and choreography in microservices?

→ Choreography: services react to events independently — no central control. Decoupled, distributed logic, hard to trace. Orchestration: central coordinator (orchestrator/saga) directs each step — centralised logic, easier to trace, couples orchestrator to services. Both valid — choose based on complexity